

Cairo University — Faculty of Computers and Artificial Intelligence

Department of Information Technology

Thaghrawy

An AI-Powered Autonomous Penetration Testing Assistant Based on the Model Context Protocol

Graduation Project — Final Documentation

Academic Year 2025–2026

Supervisor: Prof. Haitham S. Hamza

Team Members:

Name	Student ID
Loay Ahmed Badea	20220259
Youssef Ali Mohamed	20230644
Yehia Mohamed Othman	20230646
Mohamed Abd El-Nasser	20220295
Omar Ayman Mesbah	20220223
Belal Mohamed Youness	20220087

List of Figures

- Figure 4.1 — High-level system architecture
- Figure 4.2 — Class model (core entities)
- Figure 4.3 — Sequence: autonomous enumeration and auto-ingest
- Figure 4.4 — Sequence: human-in-the-loop collaboration turn
- Figure 4.5 — Data model (ChromaDB collections, engagement JSON, trajectory log)
- Figure 4.6 — Frontend three-panel wireframe

List of Tables

- Table 1.1 — Tools and technologies used
- Table 1.2 — Project timeline and status
- Table 2.1 — Comparison of related systems
- Table 3.1 — Functional requirements
- Table 3.2 — Non-functional requirements
- Table 6.1 — Representative test cases and results
- Table 6.2 — Benchmark results (live, local model)

List of Abbreviations

Abbreviation	Meaning
AI	Artificial Intelligence
AST	Average Steps per Task
CVSS	Common Vulnerability Scoring System
DPO	Direct Preference Optimization
DREAD	Damage, Reproducibility, Exploitability, Affected users, Discoverability
ESR	Exploit Success Rate
FP	False Positive
HITL	Human-in-the-Loop

Abbreviation	Meaning
LLM	Large Language Model
MCP	Model Context Protocol
OWASP	Open Worldwide Application Security Project
ReAct	Reason + Act (agent loop pattern)
SFT	Supervised Fine-Tuning
SQLi / XSS / SSRF	SQL Injection / Cross-Site Scripting / Server-Side Request Forgery

Abstract (English)

Penetration testing is essential to modern cyber-defence, but it is costly, slow, and expertise-bound, while AI-assisted alternatives are either passive advisors that still require a human to run every tool, or closed-source autonomous platforms inaccessible to the research community. Thaghrawy is an open-source, AI-powered autonomous penetration-testing assistant that addresses this gap. It wraps around 29 professional security tools (nmap, nuclei, sqlmap, dalfox, wapiti, and others) as Model Context Protocol (MCP) servers so any compatible Large Language Model can invoke them through a standard interface. A ReAct-style agent reconnoitres a target, interprets tool output, and stores findings in a ChromaDB semantic-memory store for cross-engagement recall. Engagements proceed in phases: the agent enumerates autonomously and deterministically auto-ingests low-risk findings, then hands off to a human-in-the-loop collaboration mode in which every tool call can be approved, rejected, or edited. Two reports are generated — a technical report (CVSS scores, reproduction steps) and an executive report (DREAD risk, business impact). The system is validated by 493 automated tests and a live benchmark harness measuring Exploit Success Rate, Average Steps per Task, false-positive rate, and OWASP detection rate. Every human decision is exportable as fine-tuning data (supervised examples and preference pairs), turning each engagement into knowledge that can improve future models.

المخلص (Arabic)

يُعدّ اختبار الاختراق من أهم وسائل الدفاع السيبراني الحديث، إلا أنه مكلف وبطيء ويعتمد على خبرة الأفراد، بينما تنقسم الأدوات المدعومة بالذكاء الاصطناعي بين مرشدٍ سلبي لا يزال يتطلب من الإنسان تشغيل كل أداة يدويًا، ومنصاتٍ مغلقة المصدر لا يمكن للمجتمع البحثي الوصول إليها. يقدم مشروع «تَغْرَاوي» مساعدًا مفتوح المصدر لاختبار الاختراق مدعومًا بالذكاء الاصطناعي (مثل) لمعالجة هذه الفجوة. يقوم النظام بتغليف نحو 29 أداة أمنية احترافية على هيئة خوادم تتبع بروتوكول سياق النموذج (nmap وnuclei وsqlmap وdalfox وwapiti) بحيث يستطيع أي نموذج لغوي كبير متوافق استدعاءها عبر واجهة موحدة. (MCP) بالاستطلاع وتفسير مخرجات الأدوات وحفظ النتائج في ReAct يقوم وكيلٌ ذكي بأسلوب لاستدعائها عبر المهام المختلفة. تسير المهمة على (ChromaDB) ذاكرة دلالية

مراحل: يستطلع الوكيل تلقائيًا ويستخرج النتائج منخفضة الخطورة، ثم يسلم القيادة إلى وضع تعاوني يتحكم فيه الإنسان، حيث يمكن الموافقة على كل CVSS درجات) استدعاء أداة أو رفضه أو تعديله. يُنتج النظام تقريرين: تقنيًا تم التحقق من (والأثر على العمل DREAD مخاطر) وإداريًا (خطوات إعادة الإنتاج النظام عبر 493 اختبارًا آليًا ومنظومة قياس حيّة، كما تُصدّر قرارات الإنسان كبيانات تدريب لتحسين النماذج مستقبلاً.

Chapter 1: Introduction

1.1 Background and Motivation

Modern organisations depend on web applications and APIs that grow more complex and more exposed every year. Penetration testing — proactively finding and demonstrating exploitable vulnerabilities before attackers do — is among the most effective defensive practices, but it remains expensive, time-consuming, and dependent on scarce expert talent. A junior analyst may spend days on an assessment an expert finishes in hours, and the global shortage of security professionals continues to widen.

Recent Large Language Models (LLMs) can reason about multi-step technical tasks, interpret command-line tool output, and select and sequence tools toward a goal. Applying this to penetration testing promises shorter assessments, a lower expertise threshold, and more consistent methodology. The missing piece is a **structured, standard, and safe** way for an LLM to drive real security tools — which the Model Context Protocol (MCP) now provides.

1.2 Problem Definition

Existing AI-assisted security tooling has three shortcomings. First, **passive advisors** such as PentestGPT guide a human through a methodology but never act — the human still runs every tool. Second, **autonomous commercial platforms** such as XBOW are effective but closed-source, creating a reproducibility gap for research and education. Third, most LLM-to-tool integrations are **ad-hoc**, limiting transparency and extensibility. No accessible system combines: (a) autonomous, context-driven tool invocation; (b) interpretation of unstructured tool output into structured findings; (c) memory of past engagements; and (d) meaningful human control without crippling the workflow. Thaghray targets all four.

1.3 Project Objectives and Proposed Solution

Objectives. (1) Build an LLM agent that autonomously orchestrates ~29 real security tools across the assessment lifecycle; (2) provide persistent semantic memory for cross-engagement recall; (3) generate dual technical/executive reports with CVSS and DREAD; (4) enforce safety

via registry-level exploit gating and a human approval gate; (5) evaluate accuracy with ESR, AST, FP-rate, and OWASP detection-rate benchmarks.

Proposed solution. Security tools are wrapped as MCP servers. A ReAct agent reasons, invokes a tool, observes the result, and decides the next step; confirmed findings are embedded and stored in ChromaDB. Engagements run in three phases — autonomous enumeration (with deterministic auto-ingest of easy findings), human-in-the-loop collaboration (per-tool approve/reject/edit), and reporting.

1.4 Project Scope and Limitations

In scope: web-application testing (OWASP Top 10), network reconnaissance against authorised/lab targets, automated dual reporting, and benchmark evaluation on DVWA and Juice Shop. **Out of scope:** physical security, social engineering, unauthorised/production testing, and *custom exploit development* (the system integrates existing public tools; it does not write new exploits) and *training an AI model* (the project exports training-ready data but does not train a model). **Known limitations:** results depend on the driving LLM (the local development model is weak, §6.3); all tools must be installed (addressed by a Docker image); authenticated scanning is not yet automated.

1.5 Development Methodology

A **hybrid Waterfall–Agile** approach was used: requirements and architecture were fixed early (Waterfall) to manage the uncertainty of adopting MCP and multiple external tools, while implementation proceeded in short, test-driven iterations (Agile), each adding tool wrappers or a subsystem behind passing tests. Every change is guarded by an automated test suite (493 tests) suitable for continuous integration.

1.6 Tools and Technologies Used

Table 1.1 — Tools and technologies used.

Layer	Technology	Role
Backend / API	Python 3, FastAPI,	REST + streaming chat

Layer	Technology	Role
	WebSocket	
Agent	Custom ReAct loop (core/agent.py)	Reason–Act–Observe orchestration
Tool protocol	Model Context Protocol (MCP SDK)	Standard tool interface
Memory	ChromaDB + sentence- transformers (all-MiniLM- L6-v2)	Semantic finding/technique recall
LLM	Multi-provider (LM Studio / Anthropic / OpenAI- compatible / Ollama)	Reasoning engine
Reporting	Markdown → HTML → PDF (xhtml2pdf)	Technical + executive reports
Security tools	nmap, nuclei, subfinder, amass, httpx, ffuf, gobuster, sqlmap, nikto, dalfox, wapiti, hydra, ...	Recon + exploitation
Packaging	Docker, Docker Compose	Reproducible deployment
Testing	pytest (493 tests)	Verification

1.7 Project Timeline

Table 1.2 — Project timeline and status.

#	Task	Status
1	Requirements & architecture	Completed
2	Multi-provider LLM abstraction	Completed

#	Task	Status
3	Tool registry + recon wrappers	Completed
4	ChromaDB semantic memory	Completed
5	Engagement lifecycle + persistence	Completed
6	FastAPI REST + WebSocket + frontend	Completed
7	Expand to ~29 security tools (incl. dalfox, wapiti)	Completed
8	DREAD scoring + FR-01 mode toggle	Completed
9	Dual PDF reports	Completed
10	Human-in-the-loop collaboration system	Completed
11	Training-data export (SFT + DPO)	Completed
12	Live benchmark runs	Completed (model-bound results)
13	Final documentation + poster	In progress

1.8 Report Organization

Chapter 2 reviews related work. Chapter 3 presents system analysis (stakeholders, requirements, use cases, feasibility). Chapter 4 details the design (architecture, classes, sequences, data model, UI, security). Chapter 5 covers implementation. Chapter 6 reports testing and evaluation. Chapter 7 concludes with lessons learned and future work.

Chapter 2: Related Work

Automated offensive security has moved from fragmented shell scripts toward structured, AI-driven frameworks. This chapter analyses the most relevant systems and positions Thaghrawy against them.

Ars0n Framework [1] is an open-source, Go-based bug-bounty automation framework that orchestrates 20+ tools in a fixed, containerised pipeline. It inspired Thaghrawy’s multi-tool integration but runs a predetermined sequence rather than letting an agent choose tools dynamically.

PentestGPT [2] (He et al., 2023) uses GPT-4 to guide a human tester through a methodology and reports success rates up to ~86.5% on validation suites *when given persistent context and good tool integration* — but the human still executes every tool manually. Thaghrawy invokes tools autonomously and adds human control only as an approval gate, not as the execution mechanism.

Autonomous commercial platforms (XBOW, Penlight.ai) [3] pursue “autonomous offensive security”, validating findings through real exploitation to produce reproducible proofs-of-concept. They are effective but closed-source, creating a reproducibility gap that an open project can help close.

Model Context Protocol (MCP) [4] (Anthropic, 2024) standardises how LLMs connect to external tools via a client–server, JSON-RPC interface — the “USB-C for AI tools.” It also introduces risks (tool poisoning, indirect prompt injection), motivating least-privilege and human-in-the-loop controls. Thaghrawy adopts MCP as its integration substrate.

Supporting techniques. The **ReAct** pattern [5] interleaves reasoning and acting; **ChromaDB** [6] with sentence-transformer embeddings [7] provides semantic memory; **CVSS** [8] and the **DREAD** model [9] structure risk reporting; and **Direct Preference Optimization (DPO)** [10] enables preference-based fine-tuning from the human approve/reject/ edit signal Thaghrawy captures.

Table 2.1 — Comparison of related systems.

System	Autonomy	Open source	Persistent memory	Human-in-the-loop	Training loop
Ars0n Framework	Fixed pipeline	Yes	No	No	No
PentestGPT	Advisory only	Partly	No	Human runs all	No
XBOW / Penlight	Fully autonomous	No	Partial	Limited	Proprietary
Thaghrawy	Autonomous + gated	Yes	Yes (ChromaDB)	Full (approve/reject/edit)	Yes (SFT + DPO export)

What distinguishes Thaghrawy: it is open-source, standardises tool access via MCP, remembers findings across engagements, makes the human a first-class collaborator rather than either a bystander or the tool operator, and turns each engagement into fine-tuning data.

Chapter 3: System Analysis

3.1 Stakeholder Analysis

- **Security analyst / operator (primary):** drives engagements, approves/edits tool calls, curates findings, reads reports.
- **Engagement owner / manager:** consumes the executive (DREAD) report for risk decisions.
- **LLM provider (external system):** supplies the reasoning engine (local or hosted).
- **Development team / researchers:** maintain the system and consume the exported training data.

3.2 Functional Requirements

Table 3.1 — Functional requirements.

ID	Requirement	Priority
FR-01	Support recon-only and full-analysis engagement modes (exploit tools physically excluded in recon-only)	High
FR-02	Agent autonomously selects and invokes tools from a registry	High
FR-03	Persist findings and techniques to semantic memory	High
FR-04	Generate technical and executive PDF reports	High
FR-05	Support multiple LLM providers	Medium
FR-06	Provide REST API and	Medium

ID	Requirement	Priority
	streaming WebSocket interface	
FR-07	Run an autonomous enumeration phase that auto-ingests findings, then hand off	High
FR-08	Gate each tool call for human approve/reject/edit in collaboration phase	High
FR-09	Let the human run any tool directly and curate (edit/delete) findings	Medium
FR-10	Score engagements with ESR, AST, FP-rate, and OWASP detection rate	Medium
FR-11	Support DREAD risk scoring per finding	Medium
FR-12	Block dangerous shell commands via a configurable guardrails layer	High
FR-13	Export findings, techniques, and human decisions as fine-tuning datasets	Medium

3.3 Non-Functional Requirements

Table 3.2 — Non-functional requirements.

ID	Category	Requirement
NFR-01	Maintainability	Type hints on every function;

ID	Category	Requirement
		documented modules; automated test suite
NFR-02	Portability	Deployable as a Docker container/compose stack
NFR-03	Privacy/Cost	Run fully offline with a local LLM and local embeddings
NFR-04	Security	Respect guardrails; never store raw credentials; least privilege
NFR-05	Extensibility	New tools added as MCP wrappers without touching the agent core
NFR-06	Legal	Third-party tools used per their open-source licenses

3.4 Use Cases

Primary actors: Security Analyst, AI Agent (internal), LLM Provider (external).

- **UC-01 Start engagement** — analyst creates an engagement (name, target, scope, mode); the system builds the mode-appropriate tool registry and control channel.
- **UC-02 Autonomous enumeration** — the agent recons the target and auto-ingests findings, then emits a hand-off.
- **UC-03 Collaborate** — analyst approves/rejects/edits each proposed tool call, runs tools manually, and asks the agent for help.
- **UC-04 Curate findings** — analyst edits a finding's fields or deletes a false positive.
- **UC-05 Recall memory** — the agent retrieves similar past findings before acting.
- **UC-06 Generate reports** — the system produces technical + executive PDFs.
- **UC-07 Export training data** — the team exports SFT examples and DPO preference pairs.

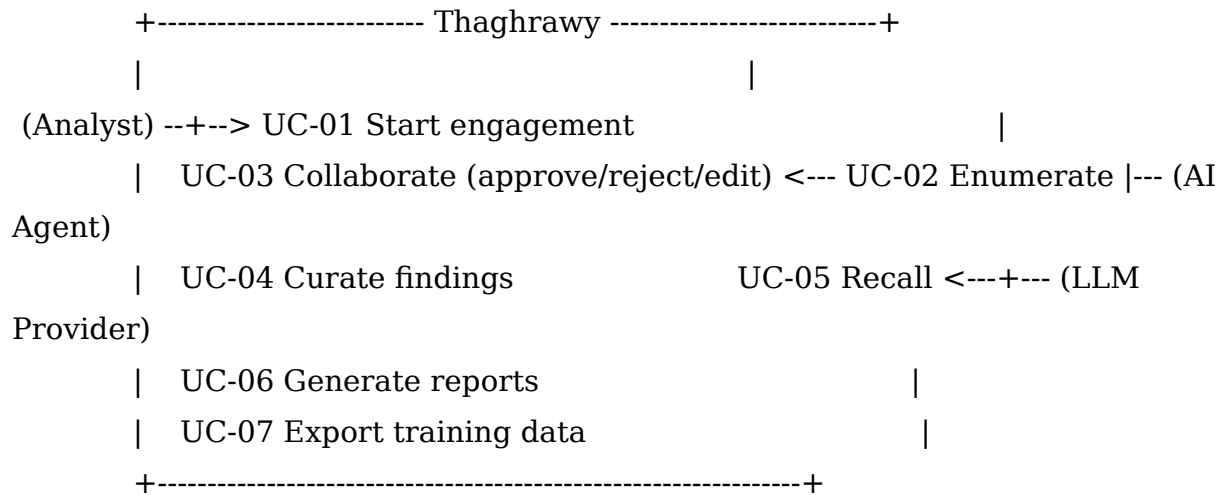


Figure 3.1 — Use-case overview (placeholder; to be replaced with a drawn UML diagram).

3.5 Feasibility Study

Technical: feasible — MCP, FastAPI, ChromaDB, and the tools are mature; the prototype already runs end-to-end with 493 passing tests. **Economic:** feasible — all components are open-source; running a local LLM and local embeddings means zero API cost, so an engagement costs only compute. **Operational:** feasible — Docker packaging removes per-machine setup; the human-in-the-loop and FR-01 gating keep operation within legal/ethical bounds.

Chapter 4: System Design

4.1 System Architecture

Thagrawy is a modular, layered system. A web frontend talks to a FastAPI backend over REST and a streaming WebSocket. The backend hosts a per-engagement PentestAgent (a ReAct loop) wired to a ToolRegistry, a MemoryStore (ChromaDB), an LLMProvider, and an AgentControl (the human-in-the-loop channel). Tools are grouped into recon and exploit MCP servers; a report server renders Markdown to PDF.

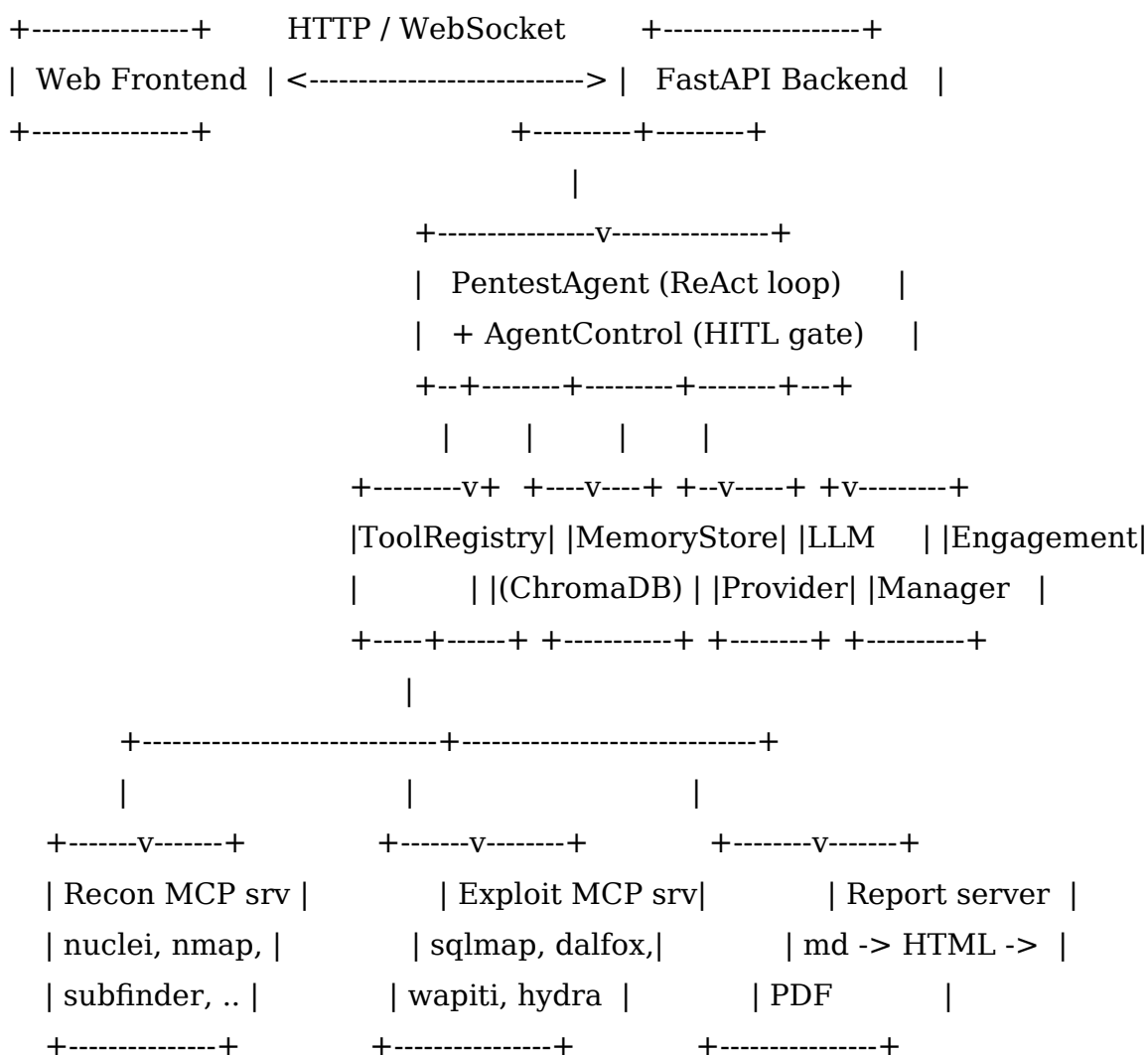
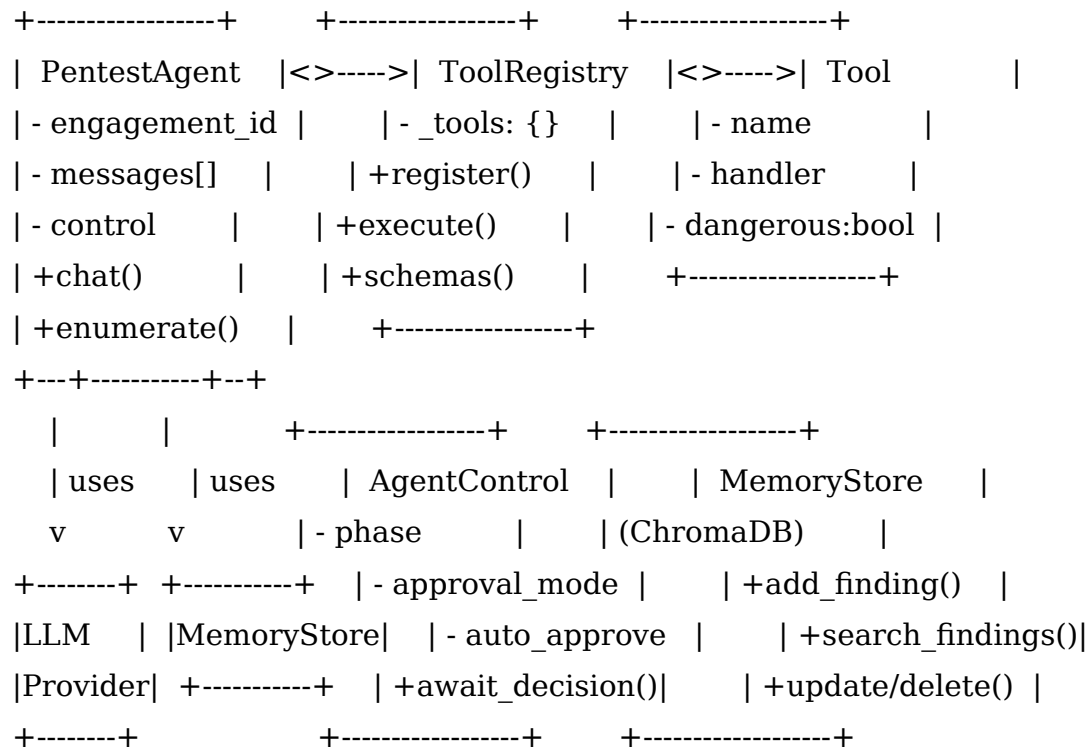


Figure 4.1 — High-level system architecture (placeholder for a drawn diagram).

4.2 Class Model (core entities)



Data models (Pydantic): Finding, Technique, Engagement

```

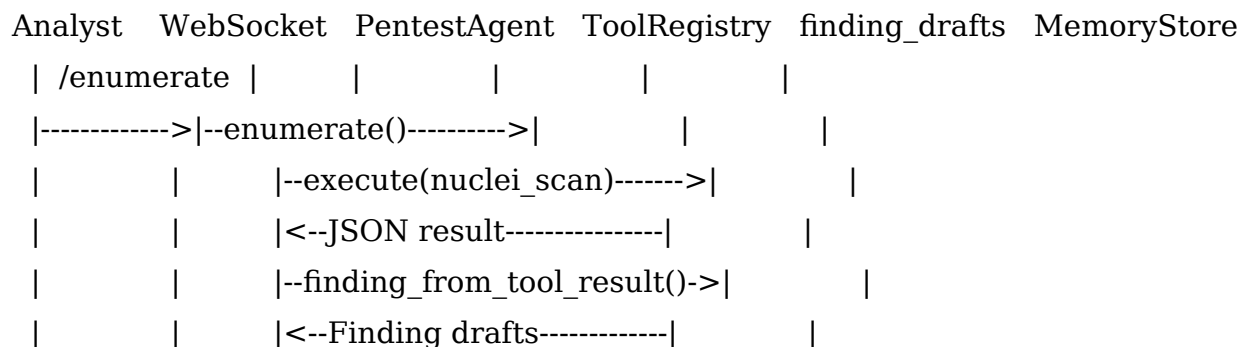
Finding{ id, title, severity, vuln_type, description, reproduction_steps,
         technique_used, target, engagement_id, date, cvss_score, dread_score, ... }

```

Figure 4.2 — Class model of core entities (placeholder).

4.3 Sequence Diagrams

Autonomous enumeration + auto-ingest (FR-07):



```

|          |          |--persist_finding()----->|
|          |<-finding_saved (xN)      |          |
|          |<-handoff (phase=collaboration)      |

```

Figure 4.3 — Enumeration and deterministic auto-ingest.

Human-in-the-loop collaboration turn (FR-08):

```

Analyst  WebSocket  PentestAgent  AgentControl  ToolRegistry
| chat msg |          |          |
|----->|--chat()->|          |
|          |          |--needs_approval()----->|
|          |<-tool_call_pending-----|          |
| approve/edit/reject |          |
|----->|--push(decision)----->|          |
|          |          |<-await_decision() resolves--|
|          |          |--execute(tool, args)----->|
|          |<-tool_result / tool_rejected          |

```

Figure 4.4 — A gated collaboration turn.

4.4 Data Model

The system uses a document/vector model rather than a relational schema, so a classical ERD does not directly apply. Three stores are used:

- **ChromaDB collections** findings and techniques — each document embedded for semantic search; metadata mirrors the Pydantic model fields.
- **Engagement JSON** — one file per engagement (engagements/sessions/<id>.json) holding identification, analysis_mode, phase, counts, and step metrics.
- **Trajectory JSONL** — one file per engagement (<id>.trajectory.jsonl) recording each human decision (proposed tool call, verdict, final arguments) for training export.

Engagement(JSON) 1 ---- * Finding(ChromaDB) Engagement 1 ---- *

TrajectoryRecord(JSONL)

```

| id, name, target, scope,          record{ tool, proposed_arguments,
| analysis_mode, phase,              verdict, final_arguments,

```

```
| findings_count, total_steps                                rejected, result }
Technique(ChromaDB){ id, name, works_against[], platform, ... }
```

Figure 4.5 — Logical data model.

4.5 User Interface

The frontend is a dark, three-panel single page: an **Engagements** panel (create/select), an **Agent Chat** panel (streamed reasoning, tool calls, and inline **Approve** / **Reject** / **Edit** controls, a phase banner, and Enumerate/Stop/Run-tool/Report/Help buttons), and a **Findings** panel (list with edit and “mark false-positive” actions, plus report links).

```
+-----+-----+-----+
| ENGAGEMENTS | AGENT CHAT  [PHASE: COLLABORATION] | FINDINGS
|
| - eng A    | [AGENT] reasoning...           | - XSS (high)  |
| - eng B    | [APPROVE?] sqlmap {...} [ok][x][e] | [edit][FP x] |
| [+ new]    | [OUT] result ... [->finding]      | REPORTS      |
|            | > type a message, or /help      | - technical.pdf |
+-----+-----+-----+
```

Figure 4.6 — Frontend three-panel wireframe (placeholder).

4.6 Security Design Considerations

Security is layered: (1) guardrails.py blocks destructive shell patterns; (2) **FR-01 registry gating** means a recon_only engagement never even registers exploit tools, so the LLM physically cannot call them; (3) the **approval gate** requires human approve/reject/edit for tool calls in the collaboration phase, giving the dangerous=True flag a real runtime effect; (4) least privilege and no-raw-credential-storage are enforced; and (5) the design explicitly accounts for MCP risks (tool poisoning, indirect prompt injection) by keeping the human in the loop for sensitive actions.

Chapter 5: Implementation

5.1 Implementation Environment and Setup

The system is Python 3 on FastAPI, packaged with Docker Compose (agent + DVWA + Juice Shop targets). The LLM defaults to a local LM Studio endpoint (OpenAI-compatible) for zero-cost, private development, and can switch to Anthropic/OpenAI/Ollama via configuration. Embeddings run locally (all-MiniLM-L6-v2). All ~29 tools are installed into the image; the ChromaDB store is a persistent Docker volume.

5.2 Key Implementation Details

- **ReAct agent (core/agent.py).** Each turn streams model tokens, collects tool calls, executes them through the registry, feeds results back, and repeats up to an iteration cap. Findings, phase changes, and human decisions are emitted as structured events.
- **MCP tool wrappers.** Recon/exploit tools are `@mcp.tool()` functions returning a **JSON string** envelope; a common helper adds timeout, workspace persistence, and error normalisation. (A subtle consequence — consumers must parse the JSON string, not assume a dict — is discussed in §7.2.)
- **Human-in-the-loop (core/control.py, api/websocket.py).** AgentControl is an asyncio control queue plus phase/auto-approve/stop state. The WebSocket runs a concurrent reader/worker: approval and stop messages flow straight to the control channel to resolve a pending prompt, while chat turns and human-run tools are serialised. The agent awaits a human decision at each gated tool call and applies approve/reject/edit.
- **Auto-ingest (core/finding_drafts.py).** Structured scanner output (nuclei, sqlmap, dalfox, wapiti, high-precision nikto lines) is mapped to Finding objects, with `vuln_type` normalised (using word-boundary matching) to the benchmark's ground-truth vocabulary so auto-ingested findings score correctly and do not inflate false positives.
- **Dual reporting (reporting/builder.py).** Pure functions build a technical report (evidence + reproduction, sorted by severity) and an executive report (business impact, DREAD tiebreak); the report server renders both to PDF.

- **Training export (training/exporter.py).** Findings/techniques become supervised examples; captured human decisions become DPO preference pairs (an edit → chosen = corrected arguments vs rejected = the model's proposal; a reject → chosen = decline).

5.3 Sample Output

Auto-ingest (live, Juice Shop): a single /enumerate produced 21 findings and handed off:

```
[TOOL] nuclei_scan {target: http://juice-shop:3000}
[FINDING_SAVED] Security Misconfiguration | nuclei: http-missing-security-headers ...
... (21 findings) ...
[HANDOFF] findings_saved=21 phase -> collaboration
```

Technical report excerpt:

```
### 1. Missing Security Headers [LOW]
- Type: Security Misconfiguration
- Technique: nuclei_scan
Description: Multiple security headers are missing (Permissions-Policy, Referrer-Policy, ...). Remediation: configure the listed headers.
```

Training export (preference pair):

```
{"prompt": "Choose the arguments for `sqlmap_scan` ...",
"chosen": "{\"url\": \"http://t/login?id=1\"}",
"rejected": "{\"url\": \"http://t\"}"}
```

Chapter 6: Testing and Evaluation

6.1 Testing Strategy

Three levels are used. **Unit tests** cover pure logic (tool wrappers with mocked subprocess, the control channel, finding-draft mapping, the exporter). **Integration tests** drive the FastAPI app and WebSocket end-to-end (help/approval/run-tool/curation routes over a real ASGI stack). A **live smoke harness** drives every registered tool against an owned target to catch real-CLI bugs that mocks cannot (a class of bug that did occur — §7.2). All **493** tests pass.

6.2 Test Cases and Results

Table 6.1 — Representative test cases and results.

ID	Scenario	Expected	Result
T-01	recon_only registry excludes exploit tools	sqlmap/nikto/hydra absent	Pass
T-02	Approve gate runs the tool	tool executes after approval	Pass
T-03	Reject feeds “[rejected by human]” and skips	no execution; model re-plans	Pass
T-04	Edit runs tool with corrected args	edited args used	Pass
T-05	Enumeration auto-ingests + flips phase	findings saved, phase=collaboration	Pass
T-06	nuclei JSON-string result → findings	drafts produced	Pass
T-07	PATCH finding severity/vuln_type	persisted, re-validated	Pass
T-08	DELETE finding	count consistent	Pass

ID	Scenario	Expected	Result
T-09	decrements count		
	Export → SFT + DPO pairs	correct schema	Pass
T-10	http_request on dead host	structured error, no crash	Pass

6.3 Performance Evaluation / Benchmarking

The harness computes the proposal’s metrics from an engagement’s saved findings.

Table 6.2 — Benchmark results (live agent run vs Juice Shop, local 9B model).

Metric	Target	Measured	Result
ESR (Exploit Success Rate)	≥ 70%	10% (1/10 categories)	Below target
AST (Average Steps per Task)	minimise	20 steps/task	—
FP-rate	≤ 15%	33% (1/3)	Below target
Detection Rate	≥ 8/10 OWASP	1/10 (A05)	Below target

Interpretation. These results are **model-bound, not pipeline-bound**. The development LLM is a weak local 9B model chosen for zero cost and privacy; it reconnoitres well but is weak at multi-step exploitation, so it reaches mostly misconfiguration findings and produced one false positive. The pipeline itself is validated end-to-end (tools run, findings save, reports generate, scoring works, and enumeration auto-ingested 21 findings). The multi-provider abstraction lets the same pipeline run on a stronger model, which is expected to move these numbers toward the targets. We report the measured values exactly.

6.4 Limitations and Known Issues

- Metric attainment depends on LLM quality (§6.3).
- Authenticated scanning (e.g. DVWA behind a login) is not yet automated.

- The /suggest and /draft LLM-assist handlers are stubbed (protocol/UI hooks exist).
- No standalone IDOR MCP server; IDOR-class coverage relies on broad scanners.

Chapter 7: Conclusion and Future Work

7.1 Summary of Achievements

Thagrawy delivers a working, open-source, MCP-based autonomous penetration-testing assistant: ~29 real tools driven by a ReAct agent; persistent semantic memory; a full human-in-the-loop collaboration model with per-tool approve/reject/edit; a phased enumeration → collaboration → reporting workflow with deterministic auto-ingest; dual CVSS/DREAD reporting; a benchmark harness; a training-data export that turns human supervision into SFT and DPO datasets; and 493 automated tests. The system meets its functional requirements and validates the full pipeline end-to-end.

7.2 Lessons Learned

- **Match the tool to the real bottleneck.** The recon bottleneck is the external tools' runtime, not the orchestration language — so Python (with its LLM ecosystem) beat a Go rewrite for our goals, while the fast Go tools are still used underneath.
- **Pick storage for the dominant query.** The defining query is semantic similarity, so ChromaDB fit better than PostgreSQL.
- **Some bugs only appear live.** A wrapper that returned a JSON *string* (not a dict) silently broke auto-ingest; mocked unit tests missed it and a live run caught it — which justified building the live smoke harness.
- **Human supervision is valuable data.** Approve/reject/edit decisions are exactly the signal needed for preference-based fine-tuning.

7.3 Future Enhancements

- Drive the pipeline with a stronger (hosted) model and re-measure the benchmarks.
- Add authenticated scanning (login → session cookie reused by scanners) to reach deeper surface (e.g. DVWA).
- Implement the `/suggest` and `/draft` LLM-assist handlers.
- Add dedicated IDOR/SSRF servers and expand OWASP coverage.
- Use the exported preference data to actually fine-tune and evaluate an improved model.

References

- [1] R-s0n, “Ars0n Framework: Automated Bug Bounty Reconnaissance,” GitHub, 2023. [Online]. Available: <https://github.com/R-s0n/ars0n-framework>
- [2] G. Deng et al., “PentestGPT: An LLM-empowered Automatic Penetration Testing Tool,” arXiv:2308.06782, 2023.
- [3] XBOW, “Autonomous Offensive Security,” 2024. [Online]. Available: <https://xbow.com>
- [4] Anthropic, “Introducing the Model Context Protocol,” 2024. [Online]. Available: <https://www.anthropic.com/news/model-context-protocol>
- [5] S. Yao et al., “ReAct: Synergizing Reasoning and Acting in Language Models,” in Proc. ICLR, 2023.
- [6] Chroma, “ChromaDB: The AI-native Open-Source Embedding Database,” 2023. [Online]. Available: <https://www.trychroma.com>
- [7] N. Reimers and I. Gurevych, “Sentence-BERT: Sentence Embeddings using Siamese BERT-Networks,” in Proc. EMNLP, 2019.
- [8] FIRST, “Common Vulnerability Scoring System (CVSS) v3.1 Specification,” 2019. [Online]. Available: <https://www.first.org/cvss>
- [9] Microsoft, “DREAD Threat-Risk Rating Model,” Microsoft Security Engineering, 2002.
- [10] R. Rafailov et al., “Direct Preference Optimization: Your Language Model is Secretly a Reward Model,” in Proc. NeurIPS, 2023.
- [11] OWASP Foundation, “OWASP Top 10:2021,” 2021. [Online]. Available: <https://owasp.org/Top10/>
- [12] ProjectDiscovery, “Nuclei: Fast and Customizable Vulnerability Scanner,” GitHub, 2023. [Online]. Available: <https://github.com/projectdiscovery/nuclei>
- [13] B. Damele and M. Stampar, “sqlmap: Automatic SQL Injection and Database Takeover Tool,” 2023. [Online]. Available: <https://sqlmap.org>

[14] hahwul, “Dalfox: Parameter Analysis and XSS Scanner,” GitHub, 2023. [Online]. Available: <https://github.com/hahwul/dalfox>

[15] Wapiti Project, “Wapiti: Web Application Vulnerability Scanner,” 2023. [Online]. Available: <https://wapiti-scanner.github.io>

[16] S. Ramírez, “FastAPI,” 2023. [Online]. Available: <https://fastapi.tiangolo.com>

[17] OWASP Foundation, “OWASP Juice Shop,” 2023. [Online]. Available: <https://owasp.org/www-project-juice-shop/>

[18] Digininja, “DVWA: Damn Vulnerable Web Application,” 2023. [Online]. Available: <https://github.com/digininja/DVWA>

Appendix A — Repository Map

- `main.py` — FastAPI app (REST, WebSocket, static frontend)
- `core/` — agent loop, LLM abstraction, tool registry, control channel, finding drafts
- `memory/` — ChromaDB store, Pydantic schemas, embeddings
- `mcp_servers/` — recon/exploit/report MCP servers and tool wrappers
- `reporting/` — technical/executive report builders
- `engagements/` — engagement lifecycle + trajectory logs
- `benchmarks/` — ESR/AST/FP/Detection scoring harness
- `training/` — training-data exporter
- `api/` — FastAPI routes and WebSocket
- `frontend/` — three-panel UI
- `tests/` — 493 automated tests

Appendix B — Tool Inventory (excerpt)

Recon/scan: nmap, masscan, naabu, amass, subfinder, assetfinder, dnsx, httpx, katana, gobuster, ffuf, arjun, whois, wafw00f, web_tech_detect, nuclei, testssl, wpscan, searchsploit, enum4linux, ssrf_test, upload_test. **Exploit:** sqlmap, nikto, hydra, dalfox, wapiti, linux_privesc_check, credential_search.

Note: all ASCII diagrams in this document are placeholders intended to be replaced with professionally drawn figures for the final printed submission.